

Assignment 1

Elliott VIGIER

27 January 2025

A. Computer Vision Part

1 Variational autoencoders

1.1 Latent Variable Models

1.2 Decoder: The Generative Part of the VAE

Question 1

To sample from a VAE, the process follows ancestral sampling.

It begins by drawing a latent variable z from the prior distribution $p(z)$, which is a standard multivariate Gaussian, $\mathcal{N}(0, I_D)$.

This sampled z is then passed through the decoder (neural network parameterized by θ), which outputs the parameters of a Bernoulli distribution for the data x .

Finally, each pixel x_m in the binary image x is sampled according to the probabilities provided by the decoder.

Question 2

Monte-Carlo sampling from $p(z_n)$ approximates $\log p(x_n)$ by averaging the likelihood $p(x_n | z_n)$ over samples drawn from the prior, then taking the logarithm of that average.

Therefore, sampling only from the prior gives no direct way to adjust latent variables toward better reconstructions of x_n .

Moreover, this approach requires a large number of samples per data point, making it computationally expensive, especially as the dimensionality z increases.

1.3 KL Divergence

Question 3

For two univariate Gaussian distributions $q(x) = \mathcal{N}(\mu_q, \sigma_q^2)$ and $p(x) = \mathcal{N}(\mu_p, \sigma_p^2)$, the KL divergence is defined as:

$$D_{KL}(q||p) = \int_{-\infty}^{+\infty} q(x) \log \frac{q(x)}{p(x)} dx$$

We will take $q(x) = \mathcal{N}(0, 1)$, so:

$$q(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$$

$$p(x) = \frac{1}{\sqrt{2\pi\sigma_p^2}} \exp\left(-\frac{(x - \mu_p)^2}{2\sigma_p^2}\right)$$

Therefor:

$$\frac{q(x)}{p(x)} = \frac{\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)}{\frac{1}{\sqrt{2\pi\sigma_p^2}} \exp\left(-\frac{(x - \mu_p)^2}{2\sigma_p^2}\right)}$$

$$\frac{q(x)}{p(x)} = \sigma_p \exp\left(-\frac{x^2}{2} + \frac{(x - \mu_p)^2}{2\sigma_p^2}\right)$$

$$\log \frac{q(x)}{p(x)} = \log \sigma_p + \left(-\frac{x^2}{2} + \frac{(x - \mu_p)^2}{2\sigma_p^2}\right)$$

Small KL divergence: We will take $p(x) = \mathcal{N}(100, 1)$.

$$\log \frac{q(x)}{p(x)} = \left(\frac{-x^2 + (x - 100)^2}{2}\right)$$

$$D_{KL}(q||p) = \int_{-\infty}^{+\infty} q(x) \log \frac{q(x)}{p(x)} dx = \int_{-\infty}^{+\infty} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) \left(\frac{-x^2 + (x - 100)^2}{2}\right) dx$$

$$D_{KL}(q||p) = \int_{-\infty}^{+\infty} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) \left(\frac{-200x + 10000}{2}\right) dx$$

$$D_{KL}(q||p) = \frac{1}{2} \int_{-\infty}^{+\infty} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) (-200x + 10000) dx$$

$$D_{KL}(q||p) = -100 \int_{-\infty}^{+\infty} \frac{x}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) dx + 5000 \int_{-\infty}^{+\infty} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) dx$$

First term:

$$\int_{-\infty}^{+\infty} \frac{x}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) dx = 0,$$

because $x \exp\left(-\frac{x^2}{2}\right)$ is an odd function, and its integral over a symmetric domain is zero.

Second term:

$$\int_{-\infty}^{+\infty} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) dx = 1,$$

because this is the normalization property of the standard normal distribution $\mathcal{N}(0, 1)$.

Thus:

$$D_{KL}(q||p) = 5000$$

Large KL divergence: We can observe that the KL divergence depends on μ_p . Therefor, we will take $p(x) = \mathcal{N}(1, 1)$.

Thus:

$$D_{KL}(q||p) = \frac{1}{2}\mu_p^2 = 0.5$$

Question 4

From Eq. 16, we have :

$$\log p(x_n) = \underbrace{E_{q(z_n|x_n)}[\log p(x_n, z_n) - \log q(z_n | x_n)]}_{\text{ELBO}} + \underbrace{KL(q(z_n | x_n) || p(z_n | x_n))}_{\geq 0}$$

Because the KL divergence is always positive, the ELBO term must be less than or equal to $\log p(x_n)$. So we have :

$$\text{ELBO} \leq \log p(x_n).$$

Therefor, it is a lower bound on $\log p(x_n)$.

Computing $\log p(x_n)$ is infeasible or very difficult because it requires integrating over the intractable posterior $p(z_n | x_n)$. It is why we maximize the ELBO, which approximates $\log p(x_n)$ using a simpler distribution $q(z_n | x_n)$. Moreover, it is also way more easy to compute.

Question 5

From Eq.(16), the lower bound consists of two terms:

$$\text{ELBO} = E_{q(z_n|x_n)}[\log p(x_n, z_n)] - KL(q(z_n | x_n) \parallel p(z_n | x_n))$$

$E_{q(z_n|x_n)}[\log p(x_n, z_n)]$, represents the expected log-likelihood. It measures how well the model under q fits the observed data x_n .

$KL(q(z_n | x_n) \parallel p(z_n | x_n))$, is the KL divergence term.

When the lower-bound is pushed upwards, it can happen in two ways:

Increasing the expected log-likelihood, meaning the model learns to better reconstruct the data.

Decreasing the KL divergence makes the approximate posterior $q(z_n | x_n)$ closer to the true posterior $p(z_n | x_n)$.

1.5 Specifying the Encoder $q_\phi(z_n | x_n)$

Question 6

The total loss:

$$L = \frac{1}{N} \sum_{n=1}^N (L_n^{\text{recon}} + L_n^{\text{reg}})$$

where:

- $L_n^{\text{recon}} = -E_{q_\phi(z_n|x_n)}[\log p_\theta(x_n | z_n)]$
- $L_n^{\text{reg}} = D_{\text{KL}}(q_\phi(z_n | x_n) \parallel p_\theta(z))$

Reconstruction Loss

The term L_n^{recon} is called the reconstruction loss because it measures how well the model reconstructs x_n from z_n . By taking the expectation under $q_\phi(z_n | x_n)$, the model focuses on the latent variables most relevant to x_n .

Regularization Loss

The term L_n^{reg} is called the regularization loss because it penalizes deviations between the learned posterior $q_\phi(z_n | x_n)$ and the prior $p_\theta(z)$.

1.6 The Reparametrization Trick

1.7 Putting things together: Building a VAE

2 Diffusion

Question 11

$$\log p(x) = \log \int p(x, z_{1:T}) dz_{1:T}$$

$$\log p(x) = \log \int \frac{p(x, z_{1:T})}{q_\phi(z_{1:T} | x)} q_\phi(z_{1:T} | x) dz_{1:T} = \log E_{q_\phi(z_{1:T} | x)} \left[\frac{p(x, z_{1:T})}{q_\phi(z_{1:T} | x)} \right]$$

Since the logarithm is a concave function, and $\frac{p(x, z_{1:T})}{q_\phi(z_{1:T} | x)} > 0$, we can apply Jensen's inequality:

$$\log p(x) = \log E_{q_\phi(z_{1:T} | x)} \left[\frac{p(x, z_{1:T})}{q_\phi(z_{1:T} | x)} \right] \geq E_{q_\phi(z_{1:T} | x)} \left[\log \frac{p(x, z_{1:T})}{q_\phi(z_{1:T} | x)} \right]$$

For a *Markovian* HVAE, we have:

$$p(x, z_{1:T}) = p(x | z_1) \prod_{t=2}^T p(z_t | z_{t-1}), \quad q_\phi(z_{1:T} | x) = q_\phi(z_1 | x) \prod_{t=2}^T q_\phi(z_t | z_{t-1})$$

Therefor,

$$\log p(x) \geq E_{q_\phi(z_{1:T} | x)} \left[\log p(x | z_1) + \sum_{t=2}^T \log p(z_t | z_{t-1}) - \log q_\phi(z_1 | x) - \sum_{t=2}^T \log q_\phi(z_t | z_{t-1}) \right]$$

Question 12

Based on the three restrictions, the VDM can be summarized as follows:

Architecture: The VDM architecture is the same as the HVAE with an additional input which is the timestep t . It is used for the reverse diffusion process.

Transformations: At each step t , the forward process involves adding noise to the data using a Gaussian distribution:

$$q(z_t | z_{t-1}) = \mathcal{N}(z_t; \sqrt{\alpha_t} z_{t-1}, (1 - \alpha_t) \mathbf{I})$$

where α_t controls the noise level. Moreover, the reverse process denoises z_t to reconstruct the original data z_0 .

Inputs: For the forward process, an image (z_0). For the reverse process, standard Gaussian noise (z_T).

Outputs: At the end of the forward process, the output is a very noisy image. The reverse process generates an image which approximates the original data.

Question 13

Reconstruction term: This term ensures that the denoising process can reconstruct the original data z_0 from z_1 . It focuses on maximizing the likelihood that the model assigns to the real data given its noisy representations.

Prior matching term: This term enforces that the denoising process produces z_T such as it matches the prior $p(z_T)$ as closely as possible, where the prior represents standard Gaussian noise.

Consistency term: It ensures that each latent variable z_t is consistent with its predecessor z_{t-1} and its successor z_{t+1} . It aligns the forward distribution $q(z_t | z_{t-1})$ with the backward conditional $p_\theta(z_t | z_{t+1})$.

Question 14

$$z_t = \sqrt{\alpha_t} z_{t-1} + \sqrt{1 - \alpha_t} \epsilon_t,$$

where $\epsilon_t \sim \mathcal{N}(0, I)$. Recursively substituting for $z_{t-1}, z_{t-2}, \dots$ we have:

$$z_t = \sqrt{\prod_{i=1}^t \alpha_i} z_0 + \sum_{i=1}^t \sqrt{(1 - \alpha_i) \prod_{j=i+1}^t \alpha_j} \epsilon_i$$

We will write:

$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, so we have:

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sum_{i=1}^t \sqrt{(1 - \alpha_i) \prod_{j=i+1}^t \alpha_j} \epsilon_i$$

Esperance:

$$E[z_t] = \sqrt{\bar{\alpha}_t} z_0 + E \left[\sum_{i=1}^t \sqrt{(1 - \alpha_i) \prod_{j=i+1}^t \alpha_j} \epsilon_i \right] = \sqrt{\bar{\alpha}_t} z_0,$$

since E is linear and $E[\epsilon_i] = 0$.

Variance:

Since z_t is a sum of independent $\epsilon_i \sim \mathcal{N}(0, I)$, the variance of the sum is the sum of the variances of each term. Moreover, $\text{Var}(\epsilon_i) = I$. So we have :

$$\text{Var}(z_t) = 0 + \sum_{i=1}^t \text{Var} \left(\sqrt{(1 - \alpha_i) \prod_{j=i+1}^t \alpha_j} \epsilon_i \right) = \sum_{i=1}^t \left[(1 - \alpha_i) \prod_{j=i+1}^t \alpha_j \right] I$$

Since:

$$\begin{aligned} \sum_{i=1}^t \left[(1 - \alpha_i) \prod_{j=i+1}^t \alpha_j \right] &= (1 - \alpha_t) + \alpha_t(1 - \alpha_{t-1}) + \alpha_t \alpha_{t-1}(1 - \alpha_{t-2}) + \dots + \prod_{j=2}^t \alpha_j (1 - \alpha_1) \\ \sum_{i=1}^t \left[(1 - \alpha_i) \prod_{j=i+1}^t \alpha_j \right] &= 1 - \alpha_t \left[1 - (1 - \alpha_{t-1}) - \alpha_{t-1}(1 - \alpha_{t-2}) - \dots - \prod_{j=2}^t \alpha_j (1 - \alpha_1) \right] \\ \sum_{i=1}^t \left[(1 - \alpha_i) \prod_{j=i+1}^t \alpha_j \right] &= 1 - \alpha_t \left[\alpha_{t-1} - \alpha_{t-1}(1 - \alpha_{t-2}) - \dots - \prod_{j=2}^t \alpha_j (1 - \alpha_1) \right] \\ &\dots \\ \sum_{i=1}^t \left[(1 - \alpha_i) \prod_{j=i+1}^t \alpha_j \right] &= 1 - \prod_{j=2}^t \alpha_j [1 - (1 - \alpha_1)] = (1 - \bar{\alpha}_t) \end{aligned}$$

Therefor:

$$\text{Var}(z_t) = (1 - \bar{\alpha}_t)I$$

Conclusion: Since any linear combination of Gaussian random variables is also Gaussian, we have $z_t \sim \mathcal{N}(\sqrt{\bar{\alpha}_t}z_0, (1 - \bar{\alpha}_t)I)$.

Question 15

From equation (28):

$$\arg \min_{\theta} D_{KL}(q(z_{t-1} | z_t, z_0) \| p_{\theta}(z_{t-1} | z_t)) = \arg \min_{\theta} \frac{1}{2\sigma_q^2(t)} \|\mu_{\theta} - \mu_q\|_2^2$$

We substitute μ_q and μ_{θ} using equations (29) and (30):

$$\arg \min_{\theta} \frac{1}{2\sigma_q^2(t)} \left\| \frac{\sqrt{\alpha_t(1 - \bar{\alpha}_{t-1})}z_t + \sqrt{\bar{\alpha}_{t-1}(1 - \alpha_t)}z_0}{1 - \bar{\alpha}_t} - \frac{\sqrt{\alpha_t(1 - \bar{\alpha}_{t-1})}z_t + \sqrt{\bar{\alpha}_{t-1}(1 - \alpha_t)}\hat{z}_{\theta}(z_t, t)}{1 - \bar{\alpha}_t} \right\|_2^2$$

$$\arg \min_{\theta} \frac{\bar{\alpha}_{t-1}(1 - \alpha_t)}{2\sigma_q^2(t)(1 - \bar{\alpha}_t)^2} \|z_0 - \hat{z}_{\theta}(z_t, t)\|_2^2$$

$$\text{Since } \sigma_q^2(t) = \frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t},$$

$$\arg \min_{\theta} D_{KL}(q(z_{t-1} | z_t, z_0) \| p_{\theta}(z_{t-1} | z_t)) = \arg \min_{\theta} \frac{\bar{\alpha}_{t-1}}{2(1 - \bar{\alpha}_{t-1})(1 - \bar{\alpha}_t)} \|z_0 - \hat{z}_{\theta}(z_t, t)\|_2^2$$

This optimization minimizes the mean-squared error between z_0 (data) and $\hat{z}_{\theta}(z_t, t)$ (prediction). It helps the model to recover the original data step by step during the reconstruction process.

2.1 Generate Image by learning the denoise

Question 16

Since $q(z_t | z_0)$ is a Gaussian of form $\mathcal{N}(z_t; \sqrt{\bar{\alpha}_t}z_0, (1 - \bar{\alpha}_t)I)$, then :

The mean is $\mu = \sqrt{\bar{\alpha}_t}z_0$ and the variance is $\sigma^2 = 1 - \bar{\alpha}_t$.

$$\text{SNR}(t) = \frac{\mu^2}{\sigma^2} = \frac{(\sqrt{\bar{\alpha}_t}z_0)^2}{1 - \bar{\alpha}_t} = \frac{\bar{\alpha}_t z_0^2}{1 - \bar{\alpha}_t}$$

Since z_0^2 is constant, it does not affect the relative scale of the SNR. So we will define the SNR as:

$$\text{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}$$

Question 17

If we combine the results from question 15 and 16 we can get:

$$\arg \min_{\theta} D_{KL}(q(z_{t-1} | z_t, z_0) \| p_{\theta}(z_{t-1} | z_t)) = \arg \min_{\theta} \frac{\text{SNR}(t) \cdot \text{SNR}(t-1)}{2 \cdot \bar{\alpha}_t} \|z_0 - \hat{z}_{\theta}(z_t, t)\|_2^2$$

The penalty for deviating from z_0 depends on $\text{SNR}(t)$ and $\text{SNR}(t-1)$. When $\text{SNR}(t)$ is high, the signal is dominant, so the model is penalized more for poor predictions. But when $\text{SNR}(t)$ is low, the noise takes over, and the penalty is reduced.

2.2 Diffusion in practice

Question 18 In the notebook !

Question 19 In the notebook !

B. Natural Language Processing Part

3 Multi-lingual Translation

3.1 Pre-normalization

Question 20

Using the chain rule:

$$\frac{\partial e}{\partial x_\ell} = \frac{\partial e}{\partial x_L} \cdot \frac{\partial x_L}{\partial x_\ell} = \frac{\partial e}{\partial x_L} \cdot \frac{\partial x_L}{\partial x_{L-1}} \cdot \frac{\partial x_{L-1}}{\partial x_{L-2}} \cdot \dots \cdot \frac{\partial x_{\ell+1}}{\partial x_\ell}$$

So,

$$\frac{\partial e}{\partial x_\ell} = \frac{\partial e}{\partial x_L} \prod_{k=\ell}^{L-1} \frac{\partial x_{k+1}}{\partial x_k}$$

Case 1: Equation (31) For the case described in Equation (31):

$$x_{k+1} = \text{LayerNorm}(x_k + A(x_k))$$

the gradient of x_{k+1} with respect to x_k is:

$$\frac{\partial x_{k+1}}{\partial x_k} = \text{LayerNorm}'(x_k + A(x_k)) (1 + A'(x_k))$$

$$\frac{\partial e}{\partial x_\ell} = \frac{\partial e}{\partial x_L} \prod_{k=\ell}^{L-1} \frac{\partial x_{k+1}}{\partial x_k} = \frac{\partial e}{\partial x_L} \prod_{k=\ell}^{L-1} (\text{LayerNorm}'(x_k + A(x_k)) (1 + A'(x_k)))$$

Case 2: Equation (32) For the case described in Equation (32):

$$x_{k+1} = x_k + A(\text{LayerNorm}(x_k))$$

the gradient of x_{k+1} with respect to x_k is:

$$\frac{\partial x_{k+1}}{\partial x_k} = 1 + A'(\text{LayerNorm}(x_k)) \text{LayerNorm}'(x_k)$$

$$\frac{\partial e}{\partial x_\ell} = \frac{\partial e}{\partial x_L} \prod_{k=\ell}^{L-1} \frac{\partial x_{k+1}}{\partial x_k} = \frac{\partial e}{\partial x_L} \prod_{k=\ell}^{L-1} (1 + A'(\text{LayerNorm}(x_k)) \text{LayerNorm}'(x_k))$$

Question 21

Post-normalization (Equation 32) ensures that the final output of each layer is always normalized. This can make it easier to interpret intermediate states, because each step produces a standardized output. It also tends to reduce the need for extensive hyperparameter tuning when the network has a moderate depth. However, the main transformation such as the self-attention still acts on unnormalized inputs. As the number of layers grows, those unbounded activations can accumulate and lead to exploding or vanishing gradients, which destabilize training.

In contrast, pre-normalization (Equation 31) applies LayerNorm to the inputs first. This approach helps maintain stable activation scales throughout each layer and promotes better gradient flow which is needed for very deep architectures. While the final outputs of each block are no longer normalized, the added stability is worth the trade-off when scaling up to networks with many layers.